- THAW A CREATURE
  - Technical Implementation Specification - 3-Week Public Launch - Canonical v1.0
  - 1. Purpose
  - 2. Core Technical Principles
  - 3. Primary Architecture
  - 4. Configuration-Driven Content
  - 5. Persistent Player Data
  - 6. Data That Must NOT Persist
  - 7. Unlimited Thawing - Data Rule
  - 8. Player Runtime State
  - 9. Warmth System
  - 10. Warmth Training
  - 11. Hearth Upgrades
  - 12. Recommended Warmth
  - 13. Equipped Creature Mounted Speed
  - 14. Speed Authority
  - 15. Creature Equip Rules
  - 16. Ice Spawn Architecture
  - 17. Ice Runtime Record
  - 18. When the Creature Result Is Rolled
  - 19. Silhouette Privacy
  - 20. Grabbing Ice
  - 21. One-Ice Carry Limit
  - 22. Carrying Ice - Carrier Commitment
  - 23. Freeze / Expedition Failure
  - 24. Disconnect While Carrying Ice
  - 25. Pen Deposit
  - 26. Thaw Record
  - 27. Thaw Progress Formula
  - 28. Offline Thawing
  - 29. Great Frost Thaw Surge
  - 30. Thaw Presentation
  - 31. Unlimited Thaw Visual Layout / Player Placement
  - 32. Ready-to-Crack Interaction
  - 33. Reveal Session
  - 34. Crack -> Crack -> Smash
  - 35. Rarity Foreshadow
  - 36. Final Reveal Transaction
  - 37. New Creature Routing

- 38. Creature Ownership
- 39. Pen Income
- 40. Selling
- 41. Frozen Archive
- 42. Great Frost Architecture
- 43. World Cycle State
- 44. Great Frost Start
- 45. Ice Field Reset
- 46. Spawn Population
- 47. Special Ice
- 48. Special Ice Roll
- 49. Special Ice Presentation
- 50. Multiplayer Ice Ownership
- 50A. Canonical Survival Pressure
- 50B. Regional Frost Sprite Warmth Damage
- 50C. Guarded Ice and Shared Threat State
- 50D. Equipped Creature Vitality / Frosted State
- 50E. Mounted Survival Attack
- 50E.1 Wilderness Contest Knockback + Mounted Consequences
- 50F. Breakable Route Obstacles
- 51. Player Bases
- 52. Pen Creature Presentation
- 53. Equipped Creature Presentation - Following / Riding
- 54. Remote Requests
- 55. Server-to-Client Events
- 56. Remote Validation
- 57. Persistence Service
- 58. Join Flow
- 59. Leave Flow
- 60. Hero Moment Technical Requirements
- 61. Creature Asset Contract
- 62. Ice Asset Contract
- 63. Great Frost Art Direction Is Presentation, Not Gameplay
- 64. Prototype Technical Version
- 65. Prototype Runtime Thawing
- 66. Prototype Validation Priorities
- 67. Critical State Invariants
- 68. Explicit Technical Non-Goals
- 69. Implementation Priority
- 70. AI Implementation Rule

◦ 71. Canonical Technical Summary

THAW A CREATURE

Technical Implementation Specification - 3-Week Public Launch - Canonical v1.0

Platform: Roblox
Target: 3-week public launch
Multiplayer Target: 4 players/server initially
Architecture: Server-authoritative, configuration-driven
Primary Design Source: Concise GDD v1.0

Revision notes

- Replaces instant wilderness Ice pickup with an authoritative, **size-scaled Grab Hold**: first-pass `1.00 sec` at `0.75x Scale` to `2.00 sec` at `5.00x Scale`, configurable and initially linear. Starting a hold does not reserve the Ice; the first valid completed hold accepted by the server wins.
- Adds **Carrier Commitment** after successful Grab: only movement, jump, Ride/Unride, DropIce, and own-Pen deposit/placement remain available. Tool/Mounted Strike, PvE/PvP attack, breakables, inventory/progression actions, and reveal interactions are rejected while carrying.
- Supersedes the old no-intentional-DropIce rule with a server-authoritative DropIce request that preserves exact Ice identity/reward/size/Special metadata and applies a short first-pass `0.75 sec` same-player regrab lock.
- Refines Wilderness Contest for mounted targets: a forced PvP knock-off deals `25% of CreatureVitalityMax`; a mounted carrier cannot be ragdolled/knocked off while mount Vitality remains above zero, but valid wilderness hits deal the same `25%` mount Vitality damage. At zero, Frosted forces Unride while the exact Ice remains carried.
- Adds authoritative Home Camp / Lobby Safe-Zone semantics: no `CurrentWarmth` drain inside the shared lobby, no automatic refill/training outside the owner's own WarmZone, and no player contest effects inside the safe zone.
- Adds Frosted Companion Orb presentation without changing ownership/allocation: Frosted remains session-only, forces Unride, blocks Ride, and rematerializes at the owner's own WarmZone/failure recovery.
- Adds seven-level Pen capacity support (`4 -> 10`) with `PenLevel`, `PenSlots[1..10]`, and current Pen footprint progression `32x24 -> 32x48`.

- Replaces fixed thaw-grid-only presentation with approved player-chosen session placement inside the Pen plus deterministic rejoin fallback; logical thaw capacity remains unlimited.
- Raises launch content support to 20 configured species and region-exclusive normal-Ice creature families.
- Allows the nine new Day-11 species to use functional placeholder meshes; final asset-specific ReferenceWeight/BaseVisualScale/SilhouetteProfile may remain provisional until the final asset audit.
- Preserves species-owned BaseIncome/BaseWalkSpeed with overlapping regional income bands and the existing per-copy size formulas.
- Updates RegionConfig/SurvivalConfig production values to the new Recommended Warmth, cold, and Frost Sprite damage curve.
- Keeps Special Ice on the existing shared lifecycle while its exact eligible-region rarity tables move to 90% Rare / 10% Legendary in Constants v1.4.
- Expands Frozen Archive launch completion to 20 species and defines Sell as a low-value cleanup path rather than a primary progression source.
- Promotes every Equipped creature into a launch mount with runtime `Following <-> Riding` states.
- Uses the existing authored creature `Seat` as the default rider anchor; optional species-specific mount presentation profiles may refine pose/camera/animation without changing gameplay.
- Makes creature `FinalWalkSpeed` apply only while Riding; Following keeps the creature Equipped while the player moves at base WalkSpeed.
- Adds Ride/Unride anywhere during exploration, including while carrying Ice; mount state is non-persistent.
- Adds a mechanically identical Mounted Strike presentation for the universal survival attack, with size-independent range/damage and no trample/contact damage.
- Promotes the validated Survival Pressure package into launch architecture.
- Locks one universal survival tool, stationary Frost Sprites, Guarded Ice, Equipped-creature Vitality/Frosted state, minimal feedback, and breakable route obstacles as production scope.
- Adds region-scaled Frost Sprite Warmth damage across all five geographic Ice tiers.
- Makes every spawned Ice one shared authoritative server object; removes private onboarding-Ice copies.
- Replaces discard/origin-return carrier failure behavior with exact-Ice drop-and-rescue on Freeze, death, reset/respawn, and disconnect.
- Locks rescue/relay as valid multiplayer behavior and adds intentional DropIce as an identity-preserving voluntary release path.
- Adds Great Frost semantics for dropped Ice and shared guardian refresh.

- Marks survival runtime state and carried/dropped wilderness Ice as non-persistent.
- Updates production authority to GDD -> Launch Technical -> Constants -> Development Plan; the finalized prototype spec remains historical validation evidence.

v0.5 Architectural Revision - Per-Copy Size Jackpot

This revision deliberately supersedes the earlier species-count ownership and rarity-only fixed Speed/Income model. Public-launch architecture must now support unique CreatureGUID copies that inherit one Ice size/weight roll. Scale is derived from that roll and modifies copy-specific Pen Income and Equipped Speed. The revision occurs before persistence implementation so the saved-data schema can be correct from the start.

1. Purpose

This document defines how the canonical game design should be implemented in Roblox. The implementation must support the core fantasy:

See it. Risk it. Bring it home. Watch it thaw. Smash it open. The technical architecture must prioritize:

1. reliable expedition gameplay;
2. secure server authority;
3. simple configuration;
4. persistence;
5. multiplayer safety;
6. rapid content addition;
7. strong presentation hooks for the five Hero Moments. The implementation should not introduce additional gameplay systems, currencies, progression tracks, or abstractions unless specified by the design documents.

2. Core Technical Principles

Server Owns Gameplay Truth

The server is authoritative for:

- Cash;
- permanent Maximum Warmth;
- Current Warmth;
- Hearth Level;
- Warmth training;
- Equipped creature;
- Movement Speed;

- creature ownership;
- Pen contents;
- passive income;
- Ice existence;
- Ice ownership;
- Ice hidden creature result;
- rarity result;
- Ice WeightMultiplier/Scale lineage;
- carried/dropped Ice state and valid world position;
- thaw progress;
- Great Frost cycle gameplay;
- Special Ice;
- Survival Pressure target/damage validation;
- Frost Sprite health/target/attack state;
- Equipped-creature session Vitality/Frosted state;
- breakable route-obstacle state;
- reveal state;
- Frozen Archive;
- selling;
- persistence.

The client handles:

- UI;
- input;
- camera;
- sounds;
- particles;
- animation;
- reveal presentation;
- low-Warmth presentation;
- Great Frost presentation;
- survival hit/telegraph presentation from server-confirmed state;
- creature follower movement;
- Pen creature roaming;
- movement-speed presentation.

The client may request actions. The client never decides their results.

Retrieval-first survival rule

Survival code must remain narrowly scoped to retrieval pressure. Do not build a generic combat, item, ability, loot, crafting, or resource framework merely because the launch now contains hostile threats and a universal tool.

Stable validated prototype modules may be reused in production. Do not rename/rewrite working code solely to make class/service names match this document.

3. Primary Architecture

```
ReplicatedStorage
|
+-- Shared
|   +-- GameConfig
|   +-- CreatureConfig
|   +-- RarityConfig
|   +-- IceConfig
|   +-- WarmthConfig
|   +-- HearthConfig
|   +-- WorldCycleConfig
|   +-- RegionConfig
|   +-- SurvivalConfig
|
+-- Remotes
|   +-- GameplayRequest
|   +-- GameplayEvent
|
+-- Assets
    +-- IceVisuals
    +-- Silhouettes
    +-- Effects
    +-- WorldEffects
    +-- UI


ServerStorage
|
+-- CreatureModels
    +-- Penguin
    +-- Rabbit
    +-- ...


ServerScriptService
|
+-- ServerBootstrap
|
+-- Services
    +-- DataService
    +-- PlayerStateService
    +-- BaseService
    +-- WarmthService
    +-- WorldCycleService
    +-- IceService
    +-- ThawService
    +-- CreatureService
    +-- SurvivalService or validated equivalent survival module


StarterPlayer
+-- StarterPlayerScripts
    +-- ClientBootstrap
    |
    +-- Controllers
        +-- InteractionController
        +-- WarmthController
        +-- WorldCycleController
```

```
            +-- ThawController
            +-- CreatureVisualController
```

```
               +-- MovementPresentationController
               +-- EffectsController
               +-- UIController


Workspace
|
+-- Wilderness
|
+-- IceSpawnPoints
|   +-- Regular
|   +-- Thick
|   +-- Ancient
|   +-- Black
|   +-- Meteor
|
+-- PlayerBases
|   +-- BaseSlot01
|   +-- BaseSlot02
|   +-- BaseSlot03
|   +-- BaseSlot04
|
+-- WorldFXAnchors
```

4. Configuration-Driven Content

Gameplay numbers must not be scattered through scripts. All balancing values belong in configuration modules.

CreatureConfig

Each species defines at minimum:

```
CreatureID
DisplayName
Rarity
ModelName
SilhouetteProfile
ReferenceWeight
BaseVisualScale
BaseIncomePerSecond
BaseWalkSpeed
```

Species now owns baseline economy/traversal identity. Rarity may still influence presentation, pools, sell baselines, and rarity treatment, but it no longer uniquely determines Pen Income or Equipped Speed.

For Day-11 launch species whose final art is not yet available, `ModelName` may point to a functional placeholder asset. Placeholder `ReferenceWeight`, `BaseVisualScale`, and `SilhouetteProfile` may be provisional for integration/presentation testing and must be finalized from the actual production asset rather than treated as locked balance data. Placeholder status never changes rarity, pool, BaseIncomePerSecond, or BaseWalkSpeed.

RarityConfig

Defines supported rarity presentation/economy tiers. The current economy supports:

```
Common
Uncommon
Rare
Epic
Legendary
```

Epic/Legendary content assignment is separate from the validity of those configured economy tiers. Do not infer creatures/pools/weights merely because the tier exists.

RarityConfig may define:

- SellValue baseline / cleanup payout. Exact values are Day-14 balance data and must remain economically secondary to Pen income;
- RevealPresentationProfile;
- RarityUI;
- EffectsProfile;
- fallback migration values only when explicitly needed.

SizeConfig

Defines the one canonical size-roll system:

```
MinVisualScale
MaxVisualScale
SizeBands
WeightMultiplier ranges
WithinBandBias
IncomeScaleExponent
SpeedScaleSlope
```

Runtime formula:

```
ActualWeight = ReferenceWeight * WeightMultiplier
Scale = cbrt(ActualWeight / ReferenceWeight)
```

Because the ratio reduces to WeightMultiplier, Scale can also be derived as `cbrt(WeightMultiplier)` while retaining the explicit weight model.

IceConfig

Each Ice Type defines:

```
IceType
RarityWeights
CreaturePoolsByRarity
BaseThawDuration
CarryDrainMultiplier
PresentationProfile
ReferenceWeight
BaseVisualScale
RegionID
```

Launch types remain:

```
Regular
Thick
Ancient
Black
Meteor
```

Every spawned Ice gets a server-authoritative `WeightMultiplier`. Scale is derived from the formula and is independent of hidden rarity/species unless a later explicit design change says otherwise.

Normal geographic Ice pools are region-exclusive according to Constants. Do not automatically inherit earlier-region creatures into later Ice. Region selects the creature family; rarity selects the exceptional result within that family.

WarmthConfig / RegionConfig

WarmthConfig defines starting progression and base drain/refill behavior. RegionConfig defines:

```
RegionID
DisplayName
RecommendedWarmth
ColdMultiplier
```

Recommended Warmth is informational only and must never become an access gate.

SurvivalConfig

Production survival values must live in one small configuration surface or validated equivalent. It may contain only the approved retrieval-pressure values:

- universal-tool server hit range/cooldown/damage;
- Frost Sprite health/radius/telegraph/cooldown/projectile speed;
- per-region player Warmth damage;

- Equipped-creature Vitality and creature damage;
- authored guard-profile counts/placement metadata;
- breakable hit counts;
- presentation-safe identifiers required by the validated system.

Do not evolve this configuration into a generic weapon/enemy/item/stat framework.

HearthConfig

Each Hearth level defines:

```
Level
UpgradeCost
WarmthTrainingRate
VisualStage
```

The Hearth never defines a Maximum Warmth cap.

5. Persistent Player Data

Public launch requires persistent data. Recommended schema:

```
DataVersion = 2

Cash
MaxWarmth
HearthLevel
PenLevel

CreatureCopies = {
    [CreatureGUID] = {
        CreatureID,
        WeightMultiplier,
    }
}

PenSlots = {
    [1] = CreatureGUID or nil,
    ...
    [10] = CreatureGUID or nil,
}

Only indices `1..PenCapacity(PenLevel)` are active/earning/assignable. Locked higher indices remain
empty.

EquippedCreatureGUID = CreatureGUID or nil

DiscoveredCreatures = {
    [CreatureID] = true
}

ThawingIce = {
    {
        ThawID,
        IceType,
        IsSpecial,
        CreatureID,        -- server-only hidden reward
        Rarity,            -- server-only hidden reward
        WeightMultiplier,  -- inherited from wilderness Ice
        DepositedAtUnix,
        BaseDurationSeconds,
        BonusProgressSeconds,
    }
}

PityState = {
    CommonStreak = number
}
```

`Scale`, `FinalIncome`, and `FinalWalkSpeed` should normally be **derived**, not redundantly persisted:

```
Scale = cbrt(WeightMultiplier)
FinalIncome = f(CreatureConfig[CreatureID].BaseIncomePerSecond, Scale)
FinalWalkSpeed = g(CreatureConfig[CreatureID].BaseWalkSpeed, Scale)
```

This keeps each individual creature record compact while preserving unique jackpot identity.

Data migration from the old species-count prototype representation must be explicit. Do not silently mix count-based ownership and GUID-based ownership after the migration task is complete.

6. Data That Must NOT Persist

Do not persist:

- CurrentWarmth;
- carried Ice ownership;
- dropped wilderness-Ice ownership or world position;
- current character position;
- active reveal session;
- temporary Frozen state;
- current Great Frost phase;
- world Ice ownership;
- follower/mount presentation position;
- `IsRiding` / MountState;
- mount Seat occupancy/presentation state;
- Pen creature visual positions;
- Frost Sprite health;
- Frost Sprite current target/attack/cooldown state;
- universal-tool cooldown;
- current breakable damage/state;
- Equipped-creature current Vitality;
- Frosted/Downed state;
- current survival encounter target.

On join:

```
CurrentWarmth = MaxWarmth
CreatureVitality = full for the currently Equipped creature, if any
Frosted = false
MountState = Following if a creature is Equipped, otherwise None
```

The player starts safely at their assigned base.

A wilderness Ice becomes persistent only after successful Pen deposit converts it into a `ThawItem`.

7. Unlimited Thawing - Data Rule

There is no player-facing Thaw capacity. ThawingIce therefore contains an arbitrary number of logical Ice records. Do not create: MaxThawSlots ; purchasable thaw capacity; separate incubator slots. The persistent records must remain compact. Do not store: Models; CFrames;

visual state; particles; redundant creature metadata. Only gameplay data should persist. An internal technical safety guard may protect against impossible/exploit-generated data volumes, but this must never function as normal gameplay progression.

8. Player Runtime State

PlayerStateService maintains non-persistent session state such as:

```
CurrentWarmth
IsInsideOwnWarmZone
IsInsideHomeSafeZone
IsTrainingWarmth
CurrentlyCarryingIceID
ActiveGrabHoldIceID
GrabHoldStartedAtServerTime
DroppedIceSelfRegrabBlockedUntil
ActiveRevealSession
AssignedBase
IsFrozen
LastGameplayRequestTimes
IsPlayerContestRagdolled
PlayerContestImmunityUntil
CurrentMovementSpeed
EquippedCreatureVitality
EquippedCreatureFrosted
MountState                 -- None / Following / Riding
LastValidCarrierCFrame / last safe drop transform
```

Persistent values are accessed through the player's loaded profile.

`LastValidCarrierCFrame` is only a runtime fallback for shared-Ice drop placement. It is not saved to DataStore.

9. Warmth System

Warmth has two values:

```
1 CurrentWarmth
2 MaxWarmth
```

MaxWarmth is permanent. CurrentWarmth is session state.

Home Camp / Lobby Safe Zone While inside the authored shared Home Camp / Lobby Safe Zone but outside the player's own WarmZone:

```
CurrentWarmth change = 0
MaxWarmth training = 0
```

This remains true while carrying Ice. The Lobby is social/safe traversal space, not a refill/training source.

Wilderness While outside the Home Camp / Lobby Safe Zone:

```
CurrentWarmth -= DrainRate * dt
```

While carrying Ice in wilderness:

```
CurrentWarmth -= DrainRate
                 * CarryMultiplier
                 * dt
```

No creature directly increases Warmth.

Own Warm Base While inside the player's own WarmZone:

```
            1 CurrentWarmth += RefillRate * dt
```

up to:

```
            1 MaxWarmth
```

At the same time:

```
            1 MaxWarmth += HearthTrainingRate * dt
```

Warmth training does not stop at a cap.

10. Warmth Training

Warmth training is automatic. The player does not send:

```
            1 TrainWarmth
```

requests. The server determines:

```
            1 player is inside own WarmZone
```

and awards training continuously. The training rate is:

```
1 HearthConfig[HearthLevel].TrainingRate
```

There is: no XP bar; no Warmth currency; no diminishing-return formula; no maximum trainable value. The primary UI should simply show the Warmth number increasing. Fractional progress may be stored internally while the UI displays a rounded/floored value.

11. Hearth Upgrades

UpgradeHearth is a server-authoritative purchase. Server validates:

```
1   player has loaded data
2   player is at own base
3   next level exists
4   player has enough Cash
```

Then:

```
1 Cash -= UpgradeCost
2 HearthLevel += 1
```

The new training rate applies immediately. Hearth upgrade does not directly add Warmth unless later explicitly configured. Its primary function is:

faster permanent Warmth training.

12. Recommended Warmth

Recommended Warmth is not gameplay validation. It exists for: region signs; UI; progression communication; onboarding. Example:

```
1 ANCIENT WILDERNESS
2 Recommended Warmth: 220
```

The server does not prevent a 250-Warmth player from entering.

13. Equipped Creature Mounted Speed

Equipping a creature selects the active traversal creature but does **not** permanently replace on-foot movement speed. The Equipped creature has runtime states `Following` and `Riding`.

The specific Equipped CreatureGUID determines the mount's `FinalWalkSpeed`:

```
copy = CreatureCopies[EquippedCreatureGUID]
base = CreatureConfig[copy.CreatureID].BaseWalkSpeed
scale = cbrt(copy.WeightMultiplier)
FinalWalkSpeed = base * SizeSpeedMultiplier(scale)
```

Authoritative movement rule:

```
if MountState == Riding and creature is not Frosted:
    CurrentMovementSpeed = FinalWalkSpeed
else:
    CurrentMovementSpeed = BasePlayerWalkSpeed -- 16
```

Size affects traversal more gently than Pen Income. A maximum-scale creature should be dramatically better while Riding without turning movement into teleportation or removing Warmth strategy.

The client may amplify the sensation through presentation, but the actual movement speed is server-authoritative.

14. Speed Authority

On: character spawn; Equip; Unequip; creature replacement; Ride; Unride; Frosted/Downed transition; own-Hearth recovery; the server recalculates:

```
            1 CurrentMovementSpeed
```

and applies it to the character. The client never submits its desired Speed. Gameplay interactions must use server-observed character position, not coordinates claimed by the client.

15. Creature Equip Rules

One individual CreatureGUID may be Equipped.

Equip from Inventory:

```
Inventory CreatureGUID -> EquippedCreatureGUID
```

Equip from Pen:

```
clear Pen slot
CreatureGUID -> EquippedCreatureGUID
```

Replacing Equipped creature:

```
old EquippedCreatureGUID -> Inventory
new CreatureGUID -> EquippedCreatureGUID
```

Unequip:

```
EquippedCreatureGUID -> Inventory
```

No automatic Pen refill occurs. Equip/Unequip should occur while at the player's own base. All validation uses CreatureGUID ownership, not merely CreatureID/species count.

An Equipped creature enters `Following` state after Equip, join, or respawn.

15A. Ride / Unride Runtime State

Ride/Unride is an exploration action, not a base-allocation action.

`Ride` server validation must confirm at minimum:

- a valid `EquippedCreatureGUID` exists and belongs to the player;
- the Equipped creature is not Frosted/Downed;
- the character is alive and not Frozen/respawning;
- the player is not already Riding.

`Unride` is allowed during normal exploration whenever the character is in a valid Riding state.

Ride/Unride is allowed while carrying Ice and never changes carried-Ice identity, ownership, or state. It also never reallocates Pen/Inventory ownership or changes Pen income.

Mount state is runtime-only:

```
None       -- no Equipped creature
Following  -- Equipped creature follows; player on foot
Riding     -- player rides Equipped creature
```

Do not persist `MountState` / `IsRiding`. On join or normal respawn, an Equipped creature starts Following.

15B. Mount Presentation / Seat Contract

Every launch creature is mountable. Every production creature asset must expose its authored `Seat` (or an explicitly configured equivalent) as the default rider anchor.

The Seat is a rider/presentation anchor; do not infer a need for vehicle-style acceleration, fuel, stamina, turn-radius progression, or species-specific driving physics. The smallest robust implementation may keep authoritative character locomotion and synchronize mount presentation around it.

Optional per-species `MountProfile` data may refine:

- rider offset/rotation;

- rider pose;
- camera offset;
- mounted movement animation;
- carry-Ice presentation offset;
- mounted attack animation/presentation.

These profiles are presentation-only and must not alter traversal powers, attack damage, attack range, cooldown, or economy.

16. Ice Spawn Architecture

Do not procedurally spawn Ice inside arbitrary world volumes. Use handcrafted spawn points.

```
Workspace
+-- IceSpawnPoints
    +-- Regular
    |   +-- Spawn01
    |   +-- Spawn02
    |   +-- ...
    +-- Thick
    +-- Ancient
    +-- Black
    +-- Meteor
```

Each SpawnPoint defines where Ice is allowed to appear. This gives level design control over: visibility; route choice; silhouette framing; Warmth distance; Special Ice presentation. The game randomly chooses among designed locations, rather than randomly generating arbitrary coordinates.

17. Ice Runtime Record

Each authoritative shared wilderness Ice record contains at minimum:

```
IceID
IceType
SpawnPointID
State
OwnerUserID or nil

-- Hidden reward data
CreatureID
Rarity

-- Visible jackpot lineage
WeightMultiplier

-- Shared-world lifecycle
IsSpecial
OriginCycleIndex or nil
OriginSpawnPointID or nil
LastValidWorldCFrame or equivalent runtime transform
```

Derived:

```
Scale = cbrt(WeightMultiplier)
```

`WeightMultiplier` is rolled once when the Ice opportunity is created. It must never be rerolled during carry, carrier failure/drop, rescue by another player, deposit, thaw, or reveal.

The client may know visible Scale because the Ice visibly advertises it, but the client must not receive hidden reward identity before reveal.

All spawned Ice records represent one shared server opportunity. Do not create per-player copies.

18. When the Creature Result Is Rolled

Creature outcome is selected by the server when the wilderness Ice is spawned. Pipeline:

```
 1  Ice Type
 2      v
 3  Special/Normal Roll Profile
 4      v
 5  Bad-Luck Protection if enabled
 6      v
 7  Rarity Roll
 8      v
 9  Eligible Creature Pool
10      v
11  Creature Roll
12      v
13  Store hidden result
14      v
15  Select matching/shared SilhouetteProfile
```

This allows the physical Ice to contain an appropriate silhouette before collection.

19. Silhouette Privacy

The client may receive:

```
            1 SilhouetteProfile
```

but not:

```
            1 CreatureID
            2 Rarity
```

where possible.

Multiple creatures should be able to share or reuse silhouette profiles. The goal is:

hint without reliable exact identification.

Use the canonical SilhouetteProfile IDs/mapping from Implementation Constants. The client should receive only the profile, never exact CreatureID/Rarity before approved reveal information.

20. Grabbing Ice

Wilderness Ice uses a **continuous input hold**, not instant pickup.

The required first-pass duration is derived from the Ice's already-visible `Scale`:

```
GrabHoldMinScale   = 0.75
GrabHoldMaxScale   = 5.00
GrabHoldMinSeconds = 1.00
GrabHoldMaxSeconds = 2.00

NormalizedScale = clamp((Scale - GrabHoldMinScale) / (GrabHoldMaxScale - GrabHoldMinScale), 0, 1)
GrabHoldSeconds = lerp(GrabHoldMinSeconds, GrabHoldMaxSeconds, NormalizedScale)
```

The interpolation mode is configuration-driven; first pass is linear.

The networking implementation may use server-observed ProximityPrompt hold events or an equivalent Begin/Cancel/Complete request path, but the **server must verify the hold attempt and completion**. A client may render progress but may not authoritatively claim completion time.

Starting a Grab Hold does **not** reserve the Ice:

```
State = Available
OwnerUserID = nil
```

Multiple players may hold the same Available Ice at once. The first valid **completed** hold accepted by the server atomically wins.

During an active Grab Hold:

- player may remain on foot or Riding;
- normal movement/jump is allowed while the player remains within valid interaction range;
- Tool Swing / Mounted Strike is rejected until the hold ends;
- the player is not yet a carrier and Carrier Commitment has not begun.

Cancel/reset the hold to zero when:

- the input is released/cancelled;
- server-measured Grab range is lost;
- player becomes contest-ragdolled/knocked away;
- player freezes, dies, resets, or becomes otherwise invalid;
- the Ice leaves `Available` state;
- Great Frost/world-state rules invalidate the opportunity;
- another authoritative state transition makes the attempt impossible.

At completion the server revalidates at minimum:

- Ice exists;
- `State == Available`;
- player is alive and not Frozen;
- player is not contest-ragdolled;
- player carries no other Ice;
- server-measured proximity remains valid;
- the configured hold duration was satisfied;
- Great Frost / Grab-lock state allows the claim;
- any same-player DropIce regrab lock for this Ice has expired.

Then atomically:

```
State = Carried
OwnerUserID = player.UserId
CurrentlyCarryingIceID = IceInstanceID
```

Carrier Commitment begins immediately.

Hidden CreatureID, hidden rarity, Ice region/tier, and Special status do not modify hold duration. Visible Scale is the only reward-lineage input.

21. One-Ice Carry Limit

A player may carry:

exactly 0 or 1 Ice. Attempts to Grab another Ice while carrying one are rejected. There is no backpack storage for unthawed Ice. Successfully returned Ice is deposited into the Pen instead.

22. Carrying Ice - Carrier Commitment

Carrying Ice:

- increases Warmth drain outside the Home Camp / Lobby Safe Zone;
- does not reduce WalkSpeed;
- remains visually obvious;
- preserves the exact shared Ice identity and size/reward lineage.

Once `CurrentlyCarryingIceID` is non-nil, the server enforces a restricted **Carrier Commitment** action set.

Allowed while carrying:

- normal movement;
- jump;
- RideEquippedCreature / UnrideEquippedCreature, subject to normal mount/Frosted validation;
- DropIce;
- valid own-Pen deposit/placement completion.

Rejected while carrying:

- Tool Swing / Mounted Strike / any survival attack request;
- player contest attacks;
- Frost Sprite attacks;
- breakable-route attacks;
- GrabIce / starting another Grab Hold;
- Equip/Unequip and other creature-management allocation requests;
- Sell;
- UpgradeHearth / UpgradePen;
- BeginReveal / Crack;
- starting unrelated thaw-placement or other gameplay interactions that change progression/ownership/ combat state.

The server must enforce this state; hiding client buttons alone is insufficient.

22A. Intentional DropIce

`DropIce` is now an approved authoritative request.

Server validates at minimum:

- player currently carries an Ice;
- character/runtime state is valid for a voluntary Drop;
- request rate limit passes.

On success:

1. resolve a valid safe grounded drop CFrame near the carrier; when Riding, use a ground-safe carrier/mount-relative drop position rather than Seat height;
2. detach the exact Ice;
3. preserve `IceInstanceID`, IceType, CreatureID, Rarity, WeightMultiplier/Scale, Special metadata, and applicable origin/cycle metadata;
4. set `OwnerUserID = nil`;
5. set `State = Available`;
6. clear `CurrentlyCarryingIceID`;
7. start the configured same-player self-regrab lock for that Ice;
8. do **not** force Unride.

DropIce never rerolls the Ice, converts it into Inventory, refunds Warmth, or teleports it back to spawn.

Other players may begin a Grab Hold immediately if otherwise valid. The original dropper may begin a new Grab Hold only after the first-pass `0.75 sec` self-regrab lock expires.

A carry ends through:

- successful own-Pen deposit/placement;
- intentional DropIce; or
- carrier failure: Freeze, character death, Reset Character/respawn, or disconnect.

23. Freeze / Expedition Failure

When:

```
CurrentWarmth <= 0
```

server executes canonical Freeze recovery.

Result:

1. set `IsFrozen = true`;

2. if carrying Ice, atomically detach/release that exact Ice into the shared world at the carrier's last valid safe position;
3. set the Ice `OwnerUserID = nil` and `State = Available`;
4. preserve Ice hidden reward, rarity, WeightMultiplier/Scale, IceType, and Special metadata;
5. clear the player's carry state;
6. return the character to the player's own base;
7. set `CurrentWarmth = MaxWarmth`;
8. restore the currently Equipped creature from Frosted/Downed to full session Vitality;
9. remove Frozen state after reset.

If the instantaneous failure position is invalid, use the last valid grounded/safe carrier position. If no valid safe position exists, use the Ice's authored origin as the final fallback.

The same release path must be used for character death and Reset Character/respawn so those actions cannot delete, duplicate, or teleport the reward into the player's profile.

Do not remove:

- Cash;
- MaxWarmth;
- Hearth Level;
- creatures;
- Pen Ice;
- Archive progress.

24. Disconnect While Carrying Ice

Carried Ice does not persist in the player's profile.

If a player disconnects while carrying **any** Ice type, including Special Ice:

1. capture/use the last valid safe carrier transform already known by the server;
2. detach/release the exact Ice into the shared world;
3. preserve its complete hidden reward and WeightMultiplier/Scale lineage;
4. clear `OwnerUserID`;
5. set `State = Available`;
6. allow any remaining player to claim it through the normal first-valid-Grab path.

If no valid safe transform exists, use the authored origin as final fallback.

Do not save carried Ice into the player's profile. Do not apply a Special-Ice-only return-to-origin rule.

25. Pen Deposit

The dedicated Thaw Machine is removed. A player carrying Ice returns to their own Pen and deposits it. Client requests:

```
1 DepositCarriedIce()
```

Server validates: player owns the carried Ice; player is inside/near own Pen; Ice state is Carried . Then the server converts the wilderness Ice into a persistent:

```
1 ThawItem
```

inside the player’s profile.

26. Thaw Record

Persistent thaw record:

```
ThawID
IceType
IsSpecial

-- hidden reward
CreatureID
Rarity

-- visible jackpot lineage
WeightMultiplier

DepositedAtUnix
BaseDurationSeconds
BonusProgressSeconds
```

The deposited/thawing Ice visual uses the same derived Scale as the wilderness Ice. The record preserves the exact size lineage until reveal.

27. Thaw Progress Formula

Do not run one timer object per Ice.

Progress can be derived using real-world timestamps.

```
Elapsed =
    CurrentUnixTime
    - DepositedAtUnix
    + BonusProgressSeconds
```

Then:

```
Progress =
    clamp(Elapsed / BaseDurationSeconds, 0, 1)
```

Ready condition:

```
Progress >= 1
```

This supports: online progression; offline progression; Night bonuses; persistence; server restarts; without continuously updating saved countdown values.

28. Offline Thawing

Offline thawing requires no special background process. Because:

```
CurrentUnixTime - DepositedAtUnix
```

continues increasing while the player is offline. On join:

```
ThawService
```

recalculates every saved Ice. If complete:

mark/present it as READY TO CRACK . The creature is not automatically granted. The player still performs the reveal.

29. Great Frost Thaw Surge

Only players actively present when the Great Frost occurs receive the online acceleration. At Great Frost:

```
BonusProgressSeconds += GreatFrostThawSurge
```

for each unfinished ThawItem. Alternatively, the config may later define the bonus as a percentage of base duration. Ready items are unaffected. This gives:

offline = normal speed online during Great Frost = accelerated speed.

30. Thaw Presentation

Gameplay state remains server-side. Visual models may show:

```
Stage 0 - Frozen
Stage 1 - Frost clearing
Stage 2 - silhouette clearer
Stage 3 - heavily cracked
Stage 4 - Ready
```

Stages are derived from progress thresholds. Do not expose hidden rarity through these stages.

31. Unlimited Thaw Visual Layout / Player Placement

Gameplay thaw capacity is unlimited and must not rely on named gameplay slots.

For a newly deposited Ice, the client may present a placement ghost inside the owner's currently unlocked Pen footprint. The player chooses an X/Z location with mouse/touch; the server validates the requested position before finalizing presentation. Validation must ensure at minimum:

- inside the current Pen bounds;
- no overlap with another authoritative thawing-Ice presentation footprint;
- outside reserved fence/gate/banner exclusion space;
- reasonable proximity to the player's own Pen;
- the requested ThawID belongs to the player / corresponds to the current deposit flow.

Roaming creature visuals are not authoritative obstacles and do not invalidate placement.

Exact manual thaw position is **session-only presentation data** and is not persisted. On rejoin/server reconstruction, deterministic auto-layout may place existing ThawItems into valid Pen positions.

If extreme quantities create performance problems, presentation may use model pooling, distance culling, simplified LOD, deterministic packing, or visual grouping. None of these may create a gameplay thaw limit.

32. Ready-to-Crack Interaction

When progress reaches 100%:

```
State = Ready
```

A prompt appears. Client requests:

```
BeginReveal(ThawID)
```

Server validates: player owns ThawID; item is complete; player is near own Pen; player has no conflicting reveal session. Server creates:

```
1 RevealSession
```

33. Reveal Session

Runtime structure:

```
1   SessionID
2   PlayerUserID
3   ThawID
4
5   CurrentHit = 0
6   RequiredHits = 3
7
8   LastHitTime
```

It is not persisted. Disconnecting mid-reveal leaves the Ice in:

Ready state. The player can restart later.

34. Crack -> Crack -> Smash

Client requests:

```
1 Crack(SessionID)
```

Server validates: session exists; session belongs to player; expected sequence; reasonable rate limit; player remains near the Ice. Then:

```
1 CurrentHit += 1
```

Hits:

```
1 1 = Crack
2 2 = Crack
3 3 = Smash
```

No skill grade exists. No timing score exists.

35. Rarity Foreshadow

Hit #2 may provide server-approved presentation information. For example:

```
1 ForeshadowEffectProfile
```

The server may choose: none for Common; subtle for Rare; strong for Legendary.

Do not send:

```
1 Rarity = Legendary
```

until the final reveal.

36. Final Reveal Transaction

On Hit #3:

1. lock the ThawItem as resolving;
2. obtain stored `CreatureID`, `Rarity`, and `WeightMultiplier`;
3. create exactly one new `CreatureGUID` copy;
4. store `{CreatureID, WeightMultiplier}` in `CreatureCopies[CreatureGUID]`;
5. mark `DiscoveredCreatures[CreatureID] = true`;
6. route that CreatureGUID into Pen or Inventory;
7. remove the ThawItem;
8. clear reveal session;
9. send reveal event including sanitized copy identity and derived visible stats.

The creature visual must inherit the exact same Scale derived from the Ice's WeightMultiplier. There is no second size roll at reveal.

This operation must be idempotent. Repeated/duplicate Crack requests must never grant multiple creature copies.

37. New Creature Routing

After reveal:

```
if Pen has empty unlocked creature capacity for current PenLevel:
    PenSlots[firstEmptyUnlocked] = CreatureGUID
else:
    CreatureGUID remains in derived Inventory
```

No modal selection is required.

38. Creature Ownership

Launch ownership is per-copy:

```
CreatureCopies[CreatureGUID] = {
    CreatureID,
    WeightMultiplier,
}
```

Inventory is derived:

```
all owned CreatureGUIDs
- CreatureGUIDs currently in PenSlots
- EquippedCreatureGUID
```

Each copy may differ in Scale, Income, and Speed even when multiple copies share the same CreatureID.

Only the approved size/weight roll creates random per-copy stat variation. Do not add independent randomized Damage, Defense, skills, levels, affixes, or arbitrary stat rolls.

39. Pen Income

Pen income is calculated from occupied **unlocked** `PenSlots` according to current `PenLevel`/capacity using one aggregate server tick. Current cadence:

```
1 second
```

For each occupied slot:

```
copy = CreatureCopies[CreatureGUID]
base = CreatureConfig[copy.CreatureID].BaseIncomePerSecond
scale = cbrt(copy.WeightMultiplier)
FinalIncome = round(base * scale^2)
```

Then:

```
TotalIncome = sum(FinalIncome for occupied PenSlots)
Cash += TotalIncome
```

No independent timer per creature. No offline Pen income. Equipped and Inventory copies earn zero. A gigantic copy may therefore be an extreme economic jackpot.

Pen Level / Capacity contract

```
PenLevel 1 -> Capacity 4 -> 32x24
PenLevel 2 -> Capacity 5 -> 32x28
PenLevel 3 -> Capacity 6 -> 32x32
PenLevel 4 -> Capacity 7 -> 32x36
PenLevel 5 -> Capacity 8 -> 32x40
PenLevel 6 -> Capacity 9 -> 32x44
PenLevel 7 -> Capacity 10 -> 32x48
```

`UpgradePen` is a server-authoritative Cash purchase. The server validates own base/proximity, loaded profile, next level, and configured cost before deducting Cash and incrementing `PenLevel`. There is no downgrade and no Robux capacity.

Visual creature roaming is independent of logical `PenSlots`; active creatures share the entire unlocked Pen area.

40. Selling

Sell requests operate on a specific CreatureGUID. Server validates:

- player owns the GUID;
- source state is legal;
- player is at own base;
- the copy can legally be removed.

Sell from Inventory:

```
remove CreatureCopies[CreatureGUID]
Cash += configured SellValue
```

Selling is a cleanup path, not a primary economy strategy. Exact SellValue values remain Day-14 balance constants and should be intentionally small relative to the creature's recurring Pen value.

Sell Value is not automatically size-scaled by the `Scale^2` Pen-income multiplier. If selling from Pen or Equipped is supported, clear that exact GUID allocation before removal. Frozen Archive discovery remains species-based and never reverses.

41. Frozen Archive

Persistent structure:

```
DiscoveredCreatures[CreatureID] = true
```

It never becomes false. The launch Archive presents **20 species slots** (`X / 20`) and reads from this set. Selling the last owned copy does not remove discovery.

42. Great Frost Architecture

WorldCycleService controls:

```
1 Day
2 Great Frost
```

Launch uses one shared wall-clock rhythm rather than a unique timer per server. Canonical calculation:

```
1 CycleEpochUnix = 0
2 CycleIndex = floor((CurrentUnixTime - CycleEpochUnix) / CycleLength)
3 CyclePosition = (CurrentUnixTime - CycleEpochUnix) % CycleLength
4 Day if CyclePosition < (CycleLength - GreatFrostDuration)
5 GreatFrost otherwise
```

With the initial 300-second cycle and 15-second Frost, Day occupies seconds 0-284 and Great Frost occupies seconds 285-299 of each cycle.

Server boot behavior: if the server boots during Day, populate the normal Ice field immediately for the current CycleIndex; if the server boots during Great Frost, do not populate a normal field until the next Day transition; do not retroactively award a Thaw Surge merely because the server started mid-Frost; do not retroactively spawn a Special Ice merely because the current CycleIndex qualifies as a Special cycle.

Each running server tracks LastProcessedFrostCycle so a Frost transition can never apply twice.

This keeps servers synchronized and prevents server boot/server hopping from manufacturing extra Frost bonuses or Special Ice.

43. World Cycle State

The service exposes:

```
1 CycleIndex
2 CurrentPhase
3 NextTransitionUnix
4 TimeRemaining
5 LastProcessedFrostCycle -- server runtime only
```

Clients receive timestamps rather than being trusted to maintain gameplay timing themselves. Client countdowns may interpolate locally between server corrections.

44. Great Frost Start

At the actual Day -> Great Frost transition observed by a running server:

Gameplay

- set `GrabLockedDuringGreatFrost = true`;
- block new wilderness GrabIce confirmations during the configured transition;
- remove/reset available unclaimed shared wilderness Ice that belongs to the field being refreshed;
- preserve carried Ice;
- preserve deposited/Ready Ice;
- clear/rebuild Guarded-Ice Frost Sprite encounters with the new field;
- reset/rebuild configured breakable-route state with the new field where authored;
- apply the configured online thaw acceleration behavior;
- determine whether this observed Frost transition is a Special Ice cycle.

If an Ice was carried when the Frost began and its carrier fails during that already-running Frost transition, the released Ice survives the current transition. It becomes `Available` subject to Grab locking and may be removed only by a later eligible world refresh if still unclaimed.

Presentation Server sends the Great Frost event/state. Clients handle sky transition, aurora, sound, wind, frost wave, and camera-safe effects. Gameplay must not depend on effects successfully rendering.

45. Ice Field Reset

Great Frost replaces short independent Ice respawn timers as the primary shared-world refresh.

During reset:

```
eligible Available shared wilderness Ice -> removed
active Guarded-Ice encounter state -> cleared
eligible breakable route state -> reset/rebuilt
```

Then, during the Frost sequence, `IceService.populateNewField()` (or validated equivalent) selects and creates the new shared field from authored candidates.

At the return to Day:

```
GrabLockedDuringGreatFrost = false
```

Carried Ice is unaffected. Deposited Pen Ice is unaffected. An Ice released by a carrier **after** the current refresh began is not retroactively deleted by that same refresh; a future Great Frost may remove it if it remains unclaimed.

There are no private onboarding Ice objects excluded from the shared field because onboarding no longer creates private Ice.

46. Spawn Population

Each Ice type defines:

```
1 CandidateSpawnPoints
```

```
2 ActiveSpawnCount
```

At each Great Frost:

1. gather valid candidate SpawnPoints;
2. randomly select configured active positions;
3. roll hidden Ice results;
4. create world Ice. This makes each cycle feel different while keeping placements handcrafted.

47. Special Ice

Every configurable X Great Frost cycles, an actual Frost transition observed by the running server may create one Special Ice opportunity unless later balance config specifies otherwise.

Special Ice is a modifier, not necessarily a sixth standard Ice Type. Example record:

```
IceType = Black
IsSpecial = true
RollProfile = Special
OriginCycleIndex = CycleIndex
OriginSpawnPointID = SpawnPointID
```

A Special Ice is one shared, contestable Ice object. First valid completed Grab Hold accepted by the server wins.

Lifecycle:

- unclaimed Special Ice remains available until claimed or until a later Great Frost removes the shared field;
- successful Pen deposit consumes the world opportunity normally;
- if its carrier freezes, dies, resets/respawns, or disconnects before deposit, release the **same Special Ice** at the carrier's last valid safe position and make it shared/Available again;
- preserve `OriginCycleIndex` / `OriginSpawnPointID` as metadata, not as a reason to teleport the failed Ice back to origin;

- carried Special Ice obeys the global rule that carried Ice survives a Great Frost transition;
- a Special Ice released during an already-running Great Frost survives that current transition and may be removed only by a future eligible field refresh if still unclaimed.

48. Special Ice Roll

Special Ice uses a dedicated rarity profile. It must guarantee:

a high-value creature opportunity with elevated rarity. Exact implementation belongs in Constants. For example, configuration may define:

```
1 SpecialRarityWeights
2 EligibleIceTypes
```

Do not hardcode Legendary.

49. Special Ice Presentation

Special Ice may be publicly recognizable through: size; runes; glow; beam; unique sound; Great Frost announcement. Its exact creature remains hidden. The server may announce:

A Strange Ice Has Awakened It should not announce:

Nidhogg spawned.

50. Multiplayer Ice Ownership

The wilderness is shared. Every spawned Ice opportunity is one authoritative server object for the entire server.

This applies to all Ice types and modifiers:

- Regular;
- Thick;
- Ancient;
- Black;
- Meteor;
- Special Ice.

No per-player wilderness Ice copies exist.

When multiple players attempt to take one Ice:

- starting a Grab Hold does not reserve it;

- multiple players may hold the same Available Ice concurrently;
- the first valid completed hold accepted by the server wins;
- server atomically changes `Available -> Carried`;
- server sets `OwnerUserID = player.UserId`;
- Carrier Commitment begins;
- every client observes the same ownership/state change.

On intentional DropIce:

```
Carried
-> exact Ice placed at valid safe grounded position
-> OwnerUserID = nil
-> Available
-> original dropper self-regrab lock begins
-> all eligible players use a fresh Grab Hold
```

On carrier failure:

```
Carried
-> same Ice restored to shared world at failure-safe position
-> OwnerUserID = nil
-> Available
-> first valid completed Grab Hold accepted by the server wins again
```

A dropped/rescued Ice keeps its original hidden reward, rarity, WeightMultiplier/Scale, IceType, and Special metadata.

The player who successfully deposits the Ice becomes the owner of the resulting persistent `ThawItem`.

Rescue/relay and voluntary relinquish are valid emergent multiplayer behavior. `DropIce` is not a direct transfer/trade request: it returns the exact Ice to shared `Available` state where normal Grab Hold contention applies.

50A. Canonical Survival Pressure

The validated Survival Pressure package is launch-canonical.

Universal survival attack:

```
input -> one survival-attack request
-> server resolves current traversal state
-> Following/on foot: present tool swing
-> Riding: present Mounted Strike
-> server validates rate/range/target/current state
-> valid approved target receives configured effect
```

Do not create combos, blocking, heavy attacks, stamina, weapon rarity, durability, upgrades, weapon inventory, multiple weapon classes, or combat XP.

Stationary Frost Sprite:

```
Dormant -> Acquire -> Telegraph -> Attack -> Cooldown -> Acquire
```

Rules:

- stationary;
- no roaming/pathfinding/chase;
- no drops/currency/XP/levels/boss logic;
- readable telegraph;
- server-authoritative attack/projectile result;
- may target a valid player or that player's currently Equipped creature according to configured targeting rules;
- can be defeated by the universal survival attack in either on-foot or mounted presentation.

Player hit result:

```
CurrentWarmth -= RegionFrostSpriteWarmthDamage
```

There is no Player HP.

Creature hit result:

```
EquippedCreatureVitality -= configured creature damage
```

Creature Vitality is session-only and never changes ownership.

50B. Regional Frost Sprite Warmth Damage

Every geographic Ice tier supports Guarded-Ice encounters. Not every individual Ice must be guarded; authored/configured unguarded opportunities are allowed.

First-pass values:

```
Snowfield / Regular          10 Warmth per hit
Frozen Pass / Thick          100 Warmth per hit
Ancient Expanse / Ancient    4,000 Warmth per hit
Black Ice Hollow / Black     250,000 Warmth per hit
Meteor Reach / Meteor        8,000,000 Warmth per hit
```

Conceptual rule:

```
RegionFrostSpriteWarmthDamage = RegionRecommendedWarmth * 0.10
```

Use the configured region value as authority. Do not compute damage from the player's `MaxWarmth`, Ice rarity, Ice size, hidden creature, or Special flag.

Special Ice uses the damage of the region where it spawned.

50C. Guarded Ice and Shared Threat State

Frost Sprites are shared server threats attached to authored Guarded-Ice opportunities/areas, not cloned separately per player.

Multiple players may interact with the same Sprite through server-authoritative requests.

Player Warmth damage affects only the player actually hit. Equipped-creature Vitality belongs only to that player's current Equipped creature.

Exact multiplayer target-priority policy remains configurable. Do not infer aggro tables, threat meters, party ownership, assists, kill credit, or loot rights from generic combat conventions.

Great Frost clears/rebuilds active guardian encounters with the shared Ice field.

50D. Equipped Creature Vitality / Frosted State

Only the currently Equipped creature requires session Vitality. Creatures do not attack autonomously and do not own combat stats. A Mounted Strike is the player's universal survival attack presented through the mount, not pet combat AI. Pen creatures do not participate.

At zero Vitality:

```
Equipped creature -> Frosted / Downed
```

Then:

- ownership is preserved;
- the creature remains logically Equipped;
- if Riding, force Unride immediately;
- follower/mount presentation transitions into a client-side **Frosted Companion Orb/Wisp** beside the player;
- Ride requests are rejected until recovery;
- the Frosted Orb is presentation-only, remains tied to the same `EquippedCreatureGUID`, is not targetable, and takes no further Vitality damage;
- creature Vitality may reach zero through Frost Sprite damage or the approved mounted-player contest damage path; both resolve through this same Frosted state;
- mounted Speed is unavailable and player returns to base WalkSpeed;
- the player may continue the expedition;

- entering the player's **own** Hearth/WarmZone restores full Vitality, clears Frosted, rematerializes the Equipped creature from the Orb, and restores Speed;
- another player's Hearth does not restore the visitor's creature;
- Freeze/death/reset recovery also restores safely without permanent loss.

50E. Mounted Survival Attack

The player learns one survival attack input across both traversal states.

```
Following / on foot -> universal tool swing
Riding              -> Mounted Strike
```

Gameplay values are identical in both states when the player is not carrying Ice:

- same configured Frost Sprite damage/effect;
- same cooldown;
- same approved target categories;
- same server authority;
- same anti-spam/rate validation.

Carrier Commitment disables the survival attack entirely while `CurrentlyCarryingIceID` is non-nil. A carrier cannot Tool Swing, Mounted Strike, damage Frost Sprites, damage breakables, or initiate player/mount contest effects.

Mounted Strike must not scale from CreatureID, rarity, visual Scale, WeightMultiplier, `FinalWalkSpeed`, or rider height. There are no mount Damage/AttackSpeed stats and no contact/trample damage.

For Riding, the server resolves hit validation from a logical `MountedCombatOrigin` near the mount's ground/front interaction area rather than from the rider's hand/Seat height. Effective attack reach remains the canonical survival-attack reach regardless of visual mount size. Per-species visual animation may be a bite, stomp, kick, swipe, peck, headbutt, or similar, but those are cosmetic presentations of the same attack.

For giant mounts, client presentation should preserve threat readability with scale-aware camera framing and/or a subtle valid-target marker for nearby approved targets. Target-assist presentation must not auto-attack or change server target validation.

50E.1 Wilderness Contest Knockback + Mounted Consequences

The same universal attack may affect another player only as the approved wilderness Ice-contest mechanic. It is not a general damage/PvP system.

Eligibility and shared validation

At hit resolution:

- attacker != target;
- both characters valid/alive;
- both server-confirmed inside Wilderness contest space;
- attacker is **not carrying Ice**;
- attacker is not in an active Grab Hold;
- target is outside applicable contest immunity/grace;
- normal attack range/cooldown validation passes.

Home Camp / Lobby rejects all player/mount contest effects. Both players must be in Wilderness at authoritative hit resolution. If a valid hit already resolved in Wilderness and subsequent physics carries the target into Home Camp, that existing ragdoll may finish, but no new contest hit may apply there.

Mounted attacker uses the same logical `MountedCombatOrigin` and canonical effective range as other Mounted Strike targets. Mount species, rarity, Scale, WeightMultiplier, `FinalWalkSpeed`, current velocity, rider height, and visual mass never increase PvP reach, damage, ragdoll, or knockback. Landing a valid hit does not force the attacker to Unride.

Target: on foot, not carrying Ice

On a valid hit:

1. cancel/reset any active Grab Hold;
2. set server-authoritative contest ragdoll;
3. apply configured knockback;
4. reject Grab completion while ragdolled;
5. recover after configured ragdoll duration;
6. apply configured post-recovery contest immunity.

No Player HP, CurrentWarmth, CreatureVitality, Cash, ownership, Inventory, Pen allocation, or Ice state changes occur.

Target: Riding, not carrying Ice

On a valid hit:

1. cancel/reset any active Grab Hold;
2. apply `PlayerContestMountDamageFraction * CreatureVitalityMax` to the target's currently Equipped creature;
3. force Unride as a consequence of the validated contest hit;
4. if Vitality remains above zero, transition the exact Equipped creature to `Following`;
5. ragdoll and knock back only the player;

6. after recovery, player remains on foot; there is no automatic remount;
7. if Vitality reaches zero, resolve normal Frosted/Downed behavior and Orb presentation;
8. apply configured post-recovery contest immunity.

The mounted contest damage is tied specifically to a **validated PvP forced-Unride event**. Manual Unride, Seat repair/cleanup, respawn, teleport/reset presentation repair, or Frosted's own forced-Unride must never apply it.

Target: on foot, carrying Ice

Carrier Commitment already disables the carrier's attack actions. Player contest effects against an on-foot carrier are rejected: no ragdoll, knockback, Warmth damage, or forced Ice drop.

Target: Riding, carrying Ice

A valid Wilderness contest hit may hinder the carrier **through the mount only**.

On success:

1. do not ragdoll the player;
2. do not apply player knockback;
3. do not force Unride while mount Vitality remains above zero;
4. do not modify CurrentWarmth;
5. do not modify Ice ownership/state and never force DropIce;
6. apply `PlayerContestMountDamageFraction * CreatureVitalityMax` to the currently Equipped creature;
7. immediately apply the configured carrier/mount contest grace so simultaneous attackers cannot collapse multiple hits into one frame;
8. if Vitality reaches zero, resolve Frosted/Downed -> force Unride -> Frosted Companion Orb; the player continues carrying the exact Ice on foot.

A full-health creature with the canonical `0.25` fraction therefore normally requires four separated valid PvP hits to reach zero. The damage uses the existing session-only `CreatureVitality`; do not create separate PvP mount HP.

Grab/Hit race

Before ownership, a valid contest hit cancels an active Grab Hold. The target must return to range and begin a fresh full hold.

The authoritative race is:

- if the completed Grab Hold transaction resolves first, Carrier Commitment begins and subsequent contest processing uses the appropriate carrier target rule above;
- if a non-carrier contest hit resolves first, the hold cancels and cannot complete while the target is ragdolled/out of valid range.

Prohibited effects

Player contest never directly changes:

- Player HP;
- CurrentWarmth;
- Cash;
- creature ownership;
- Pen/Inventory allocation;
- carried Ice identity/reward/size;
- Ice state through forced drop.

There is no trample/contact damage. Only an explicit server-validated attack request can produce the contest effect.

50F. Breakable Route Obstacles

The universal tool may also affect a small set of approved breakable environmental obstacles.

Purpose:

- create route-time decisions under Warmth pressure;
- support choices such as safe long route vs breakable shortcut vs guarded shortcut.

Breakables grant no Wood, Stone, currency, XP, enemy loot, crafting materials, or recipes. They are route objects, not resource nodes.

51. Player Bases

BaseService assigns each player one `AssignedBase`.

Launch topology uses up to four personal base plots grouped side-by-side along one shared home-camp edge. All plots face the same primary wilderness direction so players read home behind them and adventure ahead of them.

The shared home topology also contains an authored `HomeCampSafeZone` (or equivalent server-readable zone) covering the lobby/social base area.

A base contains at minimum:

- SpawnPoint;
- WarmZone;
- Hearth;
- PenArea;
- PenCreatureAnchors;

- ThawDisplayArea.

Base progression is private. Only the owner may:

- refill Current Warmth in that WarmZone;
- train Maximum Warmth there;
- upgrade its Hearth;
- manage its Pen;
- deposit Ice;
- reveal Ice;
- Equip/Unequip;
- Sell;
- restore their own currently Equipped creature from Frosted/Downed.

Inside `HomeCampSafeZone`, `CurrentWarmth` does not decrease for any player, including carriers. This safe-zone effect does not refill Current Warmth, train MaxWarmth, or recover Frosted creatures. Only the owner's own WarmZone provides those benefits. Another player's Hearth/WarmZone remains presentation/social space only for visitors. Player contest knockback is disabled throughout the Safe Zone.

Shared onboarding rule

Onboarding does not create a private Ice. A first-time player may be guided/highlighted toward a suitable **shared Regular Ice** from the same authoritative field used by everyone else.

If another player claims that Ice first, onboarding may retarget another shared Regular opportunity. If none is currently suitable, onboarding may communicate the upcoming Great Frost/world refresh rather than spawning a per-player copy.

52. Pen Creature Presentation

Pen movement remains cosmetic. Gameplay uses authoritative `PenSlots` CreatureGUID allocations, not visual position. Logical slots are invisible bookkeeping only. All active Pen creatures roam the shared unlocked `32 x Depth` area for the current PenLevel.

Each visible Pen creature should:

- use the copy's derived Scale;
- expose `SPEED <value>` and `PEN $<value>/s` in a minimal world-space label;
- remain non-authoritative for economy or ownership.

Oversized copies are allowed to visually spill outside their nominal anchor and may overlap neighboring Pen presentation. This is intentional trophy/social presentation, not extra gameplay capacity.

Do not use heavy pathfinding or server physics to make giant pets roam.

53. Equipped Creature Presentation - Following / Riding

The same Equipped copy alternates between two presentation states.

Following

Follower movement is cosmetic:

```
desired position behind player
-> smooth interpolation
-> snap/teleport if too far
```

The visible follower uses the exact equipped copy's derived Scale. The player moves on foot at base WalkSpeed while Following.

Riding

The player's character is presented at the creature's authored Seat / configured rider anchor. Movement uses the equipped copy's server-derived `FinalWalkSpeed`. Giant copies are allowed to look absurd; camera offsets should scale for readability without changing gameplay reach or collision authority.

Ride/Unride transitions must not duplicate/destroy the creature model, change the Equipped GUID, alter Pen allocation, or affect carried Ice.

Do not use follower/mount CFrame as client gameplay authority. For canonical Survival Pressure, server logical creature targeting and Mounted Strike validation derive from authoritative copy/config state rather than trusting client geometry.

54. Remote Requests

Recommended shared GameplayRequest actions:

```
ClientReady
GrabIce  -- authoritative completed-hold claim; implementation may additionally use server-observed
hold begin/cancel events
DropIce
DepositCarriedIce
BeginReveal
Crack
PlaceInPen
RemoveFromPen
EquipCreature
UnequipCreature
RideEquippedCreature
UnrideEquippedCreature
SellCreature
UpgradeHearth
UseSurvivalAttack  -- existing validated SwingSurvivalTool path may be retained internally if it
resolves both states
```

`DropIce` is an approved authoritative request. It releases the exact shared Ice; it is not a trade/transfer request.

Warmth training, income, Great Frost timing, thaw completion, Ice spawning, carrier-failure release, Frost Sprite attacks, and world reset require no trusted client request.

55. Server-to-Client Events

GameplayEvent may carry:

```
StateSnapshot
StateUpdated

CashUpdated

WarmthUpdated
WarmthTrainingUpdated

CarryStateUpdated
Frozen

IceDeposited
ThawUpdated
ThawReady

WorldCycleUpdated
GreatFrostStarted
IceFieldReset
SpecialIceSpawned

RevealStarted
CrackConfirmed
RevealCreature

PenUpdated
EquippedUpdated
MountStateUpdated
HearthUpdated

ShowMessage
SurvivalStateUpdated
FrostSpriteAttackTelegraph
SurvivalHitConfirmed
CreatureFrostedUpdated
```

These may share one RemoteEvent with action identifiers. Do not create a RemoteEvent for every button.

56. Remote Validation

Every gameplay request must validate: argument types; loaded player state; character existence; ownership; proximity where applicable; correct state; action order; rate limit; target existence. Never trust the client for: Cash; Warmth; Movement Speed; rarity; creature result; thaw completion; Night state; income; position; ownership.

57. Persistence Service

DataService is the only module that directly communicates with Roblox persistence. Responsibilities: load; defaults;

schema validation; version migration; retry; autosave; disconnect save; shutdown save; session safety. If data loading fails:

do not create a blank writable profile and overwrite legitimate data. Gameplay depending on persistent state should wait until the profile has loaded successfully.

58. Join Flow

On join:

1. DataService loads/validates the persistent profile;
2. BaseService assigns one personal base;
3. restore CreatureGUID ownership/Pen/Equipped/ThawItems;
4. derive copy Scale/Income/Speed from persisted WeightMultiplier;
5. set `CurrentWarmth = MaxWarmth`;
6. initialize Equipped-creature session Vitality to full and `Frosted = false`;
7. set MountState to `Following` when an Equipped creature exists, otherwise `None`;
8. spawn at the assigned base;
9. synchronize the current shared world/Ice field state to the client;
10. onboarding may guide an eligible new player toward an available shared Regular Ice.

Do not create a private onboarding Ice.

59. Leave Flow

On player leave/disconnect:

1. if carrying Ice, atomically release the exact Ice into the shared world at the last valid safe carrier position using the universal carrier-failure contract;
2. clear runtime carrier ownership;
3. save the persistent profile through DataService;
4. release the player's assigned base;
5. clear non-persistent survival/session state.

Do not save carried Ice, Sprite state, breakable state, creature Vitality/Frosted, or current world position.

60. Hero Moment Technical Requirements

The five Hero Moments are not optional polish concepts. The architecture must make them easy to present.

Hero Moment 1 - First Great Frost WorldCycleService supplies gameplay event. WorldCycleController + EffectsController supply: sky; aurora; wind; world sound; frost wave; spawn presentation. Effects should degrade gracefully on low-end/mobile devices.

Hero Moment 2 - Impossible-Looking Ice Primarily a Level Design requirement. Technical support: handcrafted spawn points; long-distance visibility; distinct Ice presentation; silhouette profiles. No special progression system is required.

Hero Moment 3 - Barely Made It Home Server owns Warmth.

Client receives enough Warmth state to present thresholds such as:

```
1 Low
2 Critical
3 Near-Frozen
```

Client may add: screen frost; wind intensity; audio; warning pulses. These effects never decide when the player freezes.

Hero Moment 4 - Legendary Smash Server sends reveal result once. Client selects presentation from:

```
1 RarityConfig.RevealPresentationProfile
```

Legendary presentation may include: stronger camera response; larger VFX; unique audio; creature animation; rarity reveal. Gameplay grant occurs before/independently of cosmetic completion.

Hero Moment 5 - Endgame Mounted Speed Server provides the actual mounted Speed advantage. Client adds: appropriate mount movement animation; rider pose; scale-aware camera/FOV response; trails; mount/follower presentation. A Legendary must feel faster through gameplay, not only effects.

61. Creature Asset Contract

Each creature Model must use:

```
CreatureModel
+-- Root
+-- Seat                    -- authored/default rider anchor
    +-- Visual Parts
```

Requirements:

- `PrimaryPart = Root`;
- one usable authored `Seat` or explicitly configured equivalent rider anchor;
- standardized facing direction;
- scale-safe hierarchy compatible with `Model:ScaleTo()` or equivalent uniform model scaling;
- no gameplay scripts inside creature Models;

- gameplay code references `CreatureID`, Model, Root, configured Seat/MountProfile, and copy data rather than arbitrary visual child names.

The authored model represents that species' **1.0x base visual size**. Runtime copy Scale multiplies that authored baseline and may reach 5x.

Collision/physics used for presentation should not let an enormous cosmetic creature block the player/world unexpectedly.

62. Ice Asset Contract

Each Ice visual should support presentation states without requiring procedural destruction. Preferred:

```
Stage0
Stage1
Stage2
Stage3
Ready
```

or equivalent visibility swaps. Avoid complex runtime fracture physics. The final Smash can use: prebuilt debris; particles; short-lived pieces; without requiring destructible simulation.

63. Great Frost Art Direction Is Presentation, Not Gameplay

Norse-inspired fantasy elements such as: runes; aurora; magical frost; Hearth visuals; ancient ruins; must remain primarily art/presentation. Do not introduce technical dependencies such as: rune puzzles; Viking combat; magical skill trees; mythology quests; unless separately approved.

64. Prototype Technical Version

Week-1 prototype must use the same conceptual architecture so it is not throwaway code. Prototype differences:

```
MaxPlayers = 1

Creatures = 8
Rarities = 3

IceTypes =
    Regular
    Thick

Persistence = OFF

OfflineThawing = NOT TESTABLE

World =
    Greybox

CreatureModels =
    primitive Parts

HearthLevels =
    small subset

PenCreatureSlots =
    4

GreatFrost =
    ON

GreatFrostThawSurge =
    ON

SpecialIce =
    optional / not required for core validation
```

65. Prototype Runtime Thawing

Prototype uses the same:

```
DepositedAt
BaseDuration
BonusProgress
```

logic in memory. Do not build a completely different countdown system merely because persistence is disabled. When launch persistence is added, the timestamp system should already be compatible.

66. Prototype Validation Priorities

Technical prototype is successful when the following work reliably:

```
spawn
-> Warmth
-> grab Ice
-> carry
-> freeze/fail
-> return Ice
-> Pen deposit
-> thaw
-> Night surge
-> ready
-> Crack Crack Smash
-> grant creature
-> Pen
-> income
-> Equip
-> clearly faster movement
-> Heater upgrade
-> faster Warmth training
```

Before expanding content, this complete loop should survive repeated testing.

67. Critical State Invariants

The server must always preserve:

```
0 <= CurrentWarmth <= MaxWarmth
MaxWarmth >= starting value
0 or 1 carried Ice per player
0 or 1 owner per shared wilderness Ice
0 or 1 Equipped creature
MountState in {None, Following, Riding}
active Pen creatures <= PenCapacity(PenLevel) <= 10
PenCopies + EquippedCopies <= OwnedCreatureCount
```

There is no gameplay maximum for logical `ThawingIce` count.

Shared-Ice invariants:

- one `IceID` represents one world opportunity server-wide;
- no private per-player wilderness Ice copies;
- one Ice may have only one carrier at a time;
- failure release clears owner before another Grab can succeed;
- carrier failure/rescue never rerolls CreatureID, Rarity, WeightMultiplier, Scale, IceType, or Special metadata;

- intentional DropIce is allowed only through the canonical authoritative release path and never rerolls/transfers directly;
- an Available Ice is not reserved by hold-start; only a valid completed Grab Hold may atomically create a carrier;
- while carrying, server-enforced Carrier Commitment restricts gameplay actions to move/jump/Ride-Unride/Drop/deposit-placement.

Survival invariants:

```
0 <= EquippedCreatureVitality <= CreatureVitalityMax
```

- creature Vitality is session-only;
- Frosted/Downed never changes ownership;
- `Riding` requires one valid Equipped creature and `Frosted == false`;
- Following and Riding are mutually exclusive runtime states;
- Frosted/Downed forces Unride, blocks Ride, and removes mounted Speed until own-Hearth/failure recovery;
- Player HP is not introduced;
- Frost Sprite player damage modifies CurrentWarmth only;
- Frost Sprite Warmth damage comes from configured region/location values.

Only the server may mutate gameplay truth, including Cash, MaxWarmth, HearthLevel, PenLevel, CreatureCopies, PenSlots, EquippedCreatureGUID, DiscoveredCreatures, ThawingIce, shared Ice ownership/state/reward lineage, HomeCampSafeZone membership used for authoritative rules, and survival/player-contest runtime state.

68. Explicit Technical Non-Goals

Do not implement for launch without a separate design decision:

- pickaxe/mining systems;
- creature-specific traversal abilities;
- bosses;
- roaming/pathfinding/chase enemy AI;
- large weapon/combat progression trees;
- multiple weapons/classes;
- weapon rarity/durability/upgrades/inventory;
- combos/blocking/heavy attacks/stamina;
- Player HP;
- permanent creature death;
- enemy drops/currency/XP/levels;
- autonomous pet combat AI or independent creature combat progression;

- Wood/Stone/material currencies;
- crafting/recipes/resource loops;
- separate Speed training tree;
- additional currencies;
- prestige/rebirth;
- Daily Quests/login calendars;
- randomized per-copy combat stats beyond the approved inherited size roll;
- offline Pen income;
- purchasable Thaw Slots;
- separate Thaw Machine;
- tap-to-accelerate Thawing;
- procedural Ice coordinate spawning;
- physics-heavy pet/mount AI or vehicle-progression simulation;
- complex destructible simulation.

Canonical per-copy exception: every creature copy may vary through its inherited Ice `WeightMultiplier`, which determines visual Scale and modifies Pen Income and Equipped Speed.

Canonical survival/mount exception: the narrow validated Survival Pressure package is launch scope, and the player may present the same universal attack as a Mounted Strike while Riding. Every creature is mountable, but mounts do not gain independent combat stats, autonomous attacks, or creature-specific traversal abilities.

69. Implementation Priority

If technical scope becomes constrained, protect in this order:

1. Server-authoritative expedition loop

Warmth -> Grab -> Carry -> Freeze/Return.

1. Pen thaw lifecycle

Deposit -> time -> offline progression -> Ready.

1. Reliable reveal transaction

Crack -> Crack -> Smash -> creature grant.

1. Progression

Pen income -> Hearth -> Warmth training -> Equipped creature -> Ride -> mounted Speed.

1. Great Frost

world reset + Thaw Surge.

1. Persistence

especially MaxWarmth, creatures and ThawItems.

1. Multiplayer safety

Ice ownership, bases, server validation.

1. Hero Moment presentation

Presentation can be simplified, but the five Hero Moments should remain recognizable.

70. AI Implementation Rule

When an AI coding assistant implements this project:

1. read the current task;
2. consult this Technical Specification;
3. consult Implementation Constants for exact values;
4. consult the GDD only when player-facing intent needs clarification;
5. implement only the requested task and required dependencies;
6. test;
7. fix;
8. confirm success before moving to the next task.

Do not independently add: systems; currencies; progression; libraries; remotes; abstraction layers; because they “might be useful later.” Prefer:

smallest robust implementation that satisfies the canonical design.

71. Canonical Technical Summary

The game should technically reduce to a small number of authoritative systems:

WarmthService
trains and drains expedition endurance and applies regional/hostile Warmth loss.

CreatureService
handles exact CreatureGUID ownership, Pen income, Equipped selection, Following/Riding runtime state, mounted Speed, selling, discovery, and safe integration with session-only Equipped-creature Vitality.

IceService
creates one shared wilderness field, owns exact Ice identity/reward lineage, controls first-valid Grab, carrier-failure release/rescue, and Pen conversion.

ThawService
turns successfully recovered Ice into persistent real-time anticipation.

WorldCycleService
transforms the wilderness periodically, refreshes the shared Ice/guardian field, accelerates online thawing, and creates Special Ice opportunities.

SurvivalService or validated equivalent scoped modules
validate the one universal survival attack across on-foot tool and Mounted Strike presentations, stationary Frost Sprite attacks, Guarded-Ice state, session creature Vitality/Frosted, and breakable route obstacles without becoming a generic combat framework.

BaseService
assigns private home progression spaces while preserving one shared wilderness and own-Hearth recovery rules.

DataService
ensures that the player’s creatures, Warmth, Hearth, and unfinished deposited Ice persist while transient expedition/survival world state does not.

The core architecture must protect the same player story:

```
see shared Ice
-> risk Warmth + survival pressure
-> bring it home or drop it into a rescue opportunity
-> thaw
-> CRACK -> CRACK -> SMASH
-> receive exact creature copy
-> Equip -> Follow/Ride
-> become stronger
-> go farther
```